

* FILE *

Introduction:

- All the programs that we have seen so far used input from keyboard and produce output only onto the screen or console.
- If we restricted or prohibit using only keyboard and screen as input and outputting devices then it is difficult to handle large amount of data. Also, such type of data is lost as soon as we turned off the computer i.e. this data cannot store permanently.
- To avoid these problems, the concept of file is used i.e. by using file we can store large amount of data on secondary memory devices (such as disk, magnetic tape, CD etc) permanently.

Definition:

“File is a collection of large amount of data or information which is permanently stored on secondary memory devices such as hard disk, magnetic tape etc. for future use.”

OR

“File is a place on the disk where a group of related data is stored.”

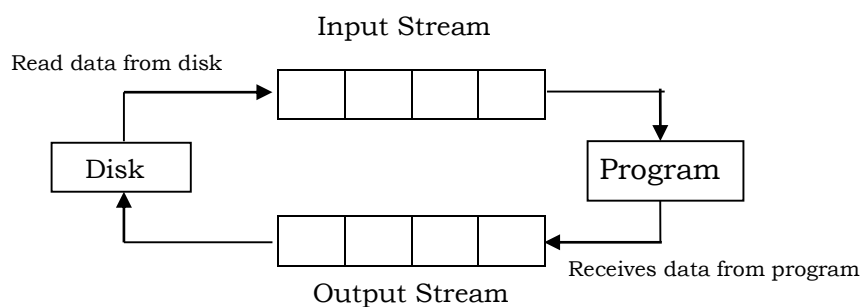
File I/O classes (File stream classes):

- To perform file related operations in C++, Three file I/O (Input/output) classes (File stream classes) were used and they are:
 - 1) ifstream class: This class used for file read (input) operation.
 - 2) ofstream class: This class used for file write (output) operation
 - 3) fstream class: This class used for both operations i.e. read and write operations.

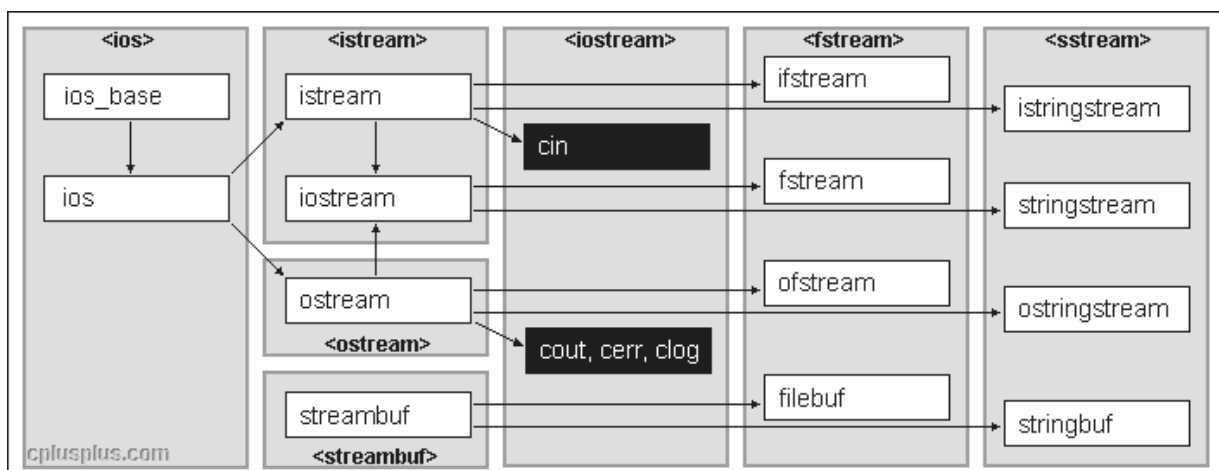
*Note that: To use any of these above class, **fstream.h** header file is used.*

Stream:

- Stream is nothing but sequence of bytes that flows from input devices to output devices and vice versa.
- A file stream is an interface between the programs and the files
- The stream that supplies data to the program is known as “input stream”
- The stream that receives the data from the program is known as “output stream”
- In short, the input stream reads data from file and output stream writes data into file.
- Following fig. shows File input-output streams:



- In C++ language, the stream class hierarchy is as follow:



- The above figure shows all stream classes available in C++ language.
- The **ios** is the base class for all stream classes. From that base class, different classes were derived which are shown in above figure

File operations:

There are different operations performed on to the file such as:

- 1) Opening the file
- 2) Reading the data from file
- 3) Writing the data into file
- 4) Closing the file.

Let us see all these operations briefly....

1) Opening the file:

- This is first operation that we applied onto the file. To read and write the data of the file, first we have to open the file. In C++ language, file can be opened by two ways:

1) Opening the file using open() member function

2) Opening the file using constructor.

I) Opening the file using open() member function:

- The file can be opened by using open() member function.
- Syntax to use open() member function:

```
FileStreamObject .open("Filename",mode);
```

Here, "FileStreamObject" is object of any file stream class
"Filename" is name of the file along with its path
"mode" is purpose for which file is opened.

File opening modes:

- Mode is nothing but purpose for which file is opened either for reading or writing or appending.

In C++ language file can be opened in three modes viz:

- 1) Read mode (ios::in)
- 2) Write mode (ios::out)
- 3) Append mode (ios::app)

I) Read mode (ios::in)

- If we want to just read the data from the file then file can be opened in read mode.
- If file is opened in read mode then only data can be read from the file where as data cannot write into opened file.

Syntax to open the file in read mode:

```
FileStreamObject .open("Filename",ios::in);
```

Here,
"FileStreamObject" is object of "ifstream" or "fstream" class
"Filename" is name of the file along with its path
"ios::in" is read mode.

E.g.

```
ifstream x; //object of "ifstream" class  
x.open("d:\\proud\\test.txt",ios::in);
```

In above example **test.txt** file which is located in directory **proud** on local disk **d** is opened in read mode.

II) Write mode (ios::out) :

- If we want to just write the data into the file then file can be opened in write mode.
- If we open the file in write mode then following things are happened:
 - 1) If opened file is not present at particular address then it is automatically created and data is writing into opened file.
 - 2) If opened file is already present at particular address then existing data of opened file is erased or deleted or cleared & just new data is writing into file.

Syntax to open the file in write mode:

```
FileStreamObject .open("Filename",ios::out);
```

Here,
"FileStreamObject" is object of "ofstream" or "fstream" class
"Filename" is name of the file along with its path
"ios::out" is write mode.

E.g.

```
ofstream x; //object of "ofstream" class  
x.open("d:\\proud\\test.txt",ios::out);
```

In above example **test.txt** file which is located in directory **proud** on local disk **d** is opened in write mode.

III) Append mode (ios::app) :

- If we want to append (attach) some data at the end of existing file then file can be opened in append mode. This mode is also used for writing purpose.

- If we open the file in append mode then following things are happened:
 - 1) If opened file is not present at particular address then it is automatically created and data is writing into opened file.
 - 2) If opened file is already present at particular address then existing data of opened file is **not** erased or **not** deleted or **not** cleared & new data is append at the end of opened file.

Syntax to open the file in append mode:

```
FileStreamObject .open("Filename",ios::app);
```

Here,

"FileStreamObject" is object of "ofstream" or "fstream" class
 "Filename" is name of the file along with its path
 "ios::app" is append mode.

E.g.

```
ofstream x; //object of "ofstream" class
x.open("d:\\proud\\test.txt",ios::app);
```

In above example **test.txt** file which is located in directory **proud** on local disk **d** is opened in append mode.

II) Opening the file using constructor:

- In C++ language, we are also able to open the file by using 'constructor' of file stream classes.
- Syntax to open the file using constructor:

```
FileStreamClass Obj("Filename", mode);
```

Here, "FileStreamClass" is name any file stream class.
 "Obj" is object of file stream class
 "Filename" is name of the file along with its path
 "mode" is purpose for which file is opened.

E.g.

```
ofstream x("d:\\proud\\test.txt",ios::out);
```

Here,

- ❖ 'x' is the object of "ofstream" class.
- ❖ We direct pass some arguments i.e. "filename" and "mode" to object 'x' therefore 'x' is parameterized constructor.
- ❖ And file 'test.txt' is opened in write mode by using constructor.

Reading & writing single character of the file:

➤ Read single character:

- To read single character from the file, *get()* member function is used.
- This *get()* member function does not accept any argument but returns one value which is nothing but reading character of the file.
- Syntax to use *get()* member function :

```
Variable=FileStreamObject .get( );
```

Here, "FileStreamObject" is object of "ifstream" or "fstream" class.
 "Variable" is variable used to hold return value from *get()*

E.g.

```
char ch;
ifstream x("d:\\proud\\test.txt",ios::in);
ch = x.get( );
```

Here, 'ch' is variable which holds reading character of "test.txt" file.

➤ Write single character:

- To write single character into the file, *put()* member function is used.
- This *put()* member function accepts one argument which is nothing but writing character and not returns value.
- Syntax to use *put()* member function :

```
FileStreamObject .put(character);
```

Here, “FileStreamObjet” is object of “ofstream” or “fstream” class.
“character” is any character that writes into file.

E.g.

```
char ch= 'A';
ofstream x("d:\\proud\\test.txt",ios::out);
x.put (ch) ;
```

Here, Character ‘A’ is write into “test.txt” file.

Reading & writing data of the file by using object of ‘ifstream’ and ‘ofstream’ class:

➤ Read the data from file by using object of ‘ifstream’ class:

- We can use directly object of ‘ifstream’ class to read the data from the file.
- Syntax:

```
ifstreamObject>>input stream;
```

Here, “ifstreamObjet” is object of “ifstream” class.

>> is the extraction operator

“input stream” may contains any variable or array or pointer etc.

Note that:

- *The syntax and use of ‘ifstream’ object is same as ‘cin’ object*
- *But ‘cin’ object read the data from key board where as ‘ifstream’ object read the data from file.*

E.g.

```
char str[50];
ifstream x("d:\\proud\\test.txt",ios::in);
x>>str; //read data from file by using ‘x’ object
```

- Here,
 - object ‘x’ is used to read the data from “test.txt” file.
 - And all reading data is stored into ‘str’ string.

➤ Write the data into file by using object of ‘ofstream’ class:

- We can use directly object of ‘ofstream’ class to write the data into the file.
- Syntax:

```
ofstreamObject<<output stream;
```

Here, “ofstreamObjet” is object of “ofstream” class.

<< is the extraction operator

“output stream” may contains any variable or array or pointer etc.

Note that:

- *The syntax and use of ‘ofstream’ object is same as ‘cout’ object*
- *But ‘cout’ object write the data onto screen or console where as ‘ofstream’ object write the data into file.*

E.g.

```
char str[] = "India Is Great";
ofstream x("d:\\proud\\test.txt",ios::out);
x<<str; //write data into file by using ‘x’ object
```

- Here,
 - Object ‘x’ is used to write data into “test.txt” file.

Closing the file:

After doing some operations (Reading, writing, appending) on file, we have to close the opened file. This work is done by ‘close()’ member function.

- Syntax:

```
FileStreamObject.close( );
```

Here, “FileStreamObjet” is object of any file stream class.

E.g.

```
ifstream x("d:\\one\\read.txt",ios::in);

.....
.....
..... } Reading operation
x.close( ); //closes the opened file
```

File error handling functions:

While doing some operations onto file, there are several possibilities to occur errors & we have to handle such errors. This task also done by some file stream class member functions and they are:

- 1) fail() 2) good() 3) bad() 4) eof()

Cause of error:

There are several things to occur errors while dealing with file and they are:

- 1) File which are attempting to open for reading does not exist.
- 2) There may not be space on the disk to store the data.
- 3) We may use an invalid name of file.
- 4) We may attempt to perform some operations when file is not opened.

To handle such errors, C++ language provides several functions & these are as follow:

1) fail() member function:

This file stream member function is used to check whether a file has been successfully opened or not for inputting or outputting.

Following example shows use of fail() member function:

```
#include<iostream.h>
#include<fstream.h>
void main( )
{
    ifstream      x( "d:\\myfile\\read.txt");
    if( x.fail( ) )    //check file is successfully opens or not
    {
        cout<< "File is not found";
    }
    ..... } Other statements
    .....
    .....
}
```

2) good() member function: This file stream member function is used to check previous file operation has been successfully done or not. Following example shows use of good() member function:

```
#include<iostream.h>
#include<fstream.h>
void main( )
{
    ofstream      x( "d:\\myfile\\write.txt");
    ..... } Write operation statements
    .....

    if( x.good( ) )
    {
        Cout<< "File writes...";
    }
}
```

3) bad() member function:

This file stream member function is used to check whether any invalid file operation has been attempted or there are unrecoverable errors or not.

Following example shows use of bad() member function:

```
#include<iostream.h>
#include<fstream.h>
void main( )
{
    ifstream      x( "d:\\myfile\\read.txt");
    if( x.bad( ) )
    {
        cout<< "File Failure" ;
    }
    ..... } Other statements
    .....
    .....
}
```

4) eof() member function:

- While reading the data from the file, it is necessary to check whether there is end of file or not.
- This eof() member function check whether file pointer reaches at the end of file or not.

Following example shows use of eof () member function:

```
#include<iostream.h>
#include<fstream.h>
void main( )
{
    ifstream      x( "d:\\myfile\\read.txt");
    while( !x.eof( ) )    //checks file pointer reaches at end of file or not
    {
        ..... } Reading statements
        .....
        .....
    }
}
```

Assignment: 05

1. What is file? Explain different file opening modes in C++
2. What is stream? Define input and out stream.
3. Draw the diagram of C++ “stream Class hierarchy”
4. What are the different file stream classes? Explain with their use.
5. How file can be opened in C++? Explain each with one example.
6. Explain following file stream member functions:
1) fail() 2) good() 3) bad() 4) fail() 5) close()
7. Write a program to read the data from file.
8. Write a program to write data into file.
9. Write a program to write and read the data of file. (Use ‘fstream’ class)
10. Write a program to copy the data of one file into another.
11. Write a program to check two files are identical or not.
12. Write a program to count total characters, total words and total lines present in file.
13. Write a program which accept any 10 numbers and write only even numbers into “even.txt” file and odd numbers into “odd.txt” file.
14. Write a program which accept 5 employee records and store all these records into file.